

Федеральное государственное автономное образовательное учреждение  
высшего образования

**«Национальный исследовательский университет ИТМО»**

Факультет программной инженерии и компьютерной техники

**Лабораторная работа №1**  
**«Принципы организации ввода/вывода**  
**без операционной системы»**

по дисциплине «Системы ввода-вывода»

Вариант: 2

**Студенты:**

Анкудинов Кирилл (1.2)

Есев Ярослав (1.2)

**Преподаватель:**

Табунщик Сергей Михайлович

# Содержание

|                                             |   |
|---------------------------------------------|---|
| 1. Введение .....                           | 2 |
| 2. Ход выполнения .....                     | 3 |
| 2.1. putchar .....                          | 3 |
| 2.2. getchar .....                          | 4 |
| 2.3. Функции по варианту .....              | 4 |
| 2.3.1. Get SBI implementation version ..... | 4 |
| 2.3.2. Hart get status .....                | 5 |
| 2.3.3. Hart stop .....                      | 5 |
| 2.3.4. System Shutdown .....                | 6 |
| 3. Результаты работы программы .....        | 7 |
| 3.1. Get SBI implementation version .....   | 7 |
| 3.2. Hart get status .....                  | 7 |
| 3.3. Hart stop .....                        | 7 |
| 3.4. System Shutdown .....                  | 8 |

# 1. Введение

**Цель:** познакомиться с принципами организации ввода/вывода без операционной системы на примере компьютерной системы на базе процессора с архитектурой RISC-V и интерфейсом OpenSBI с использованием эмулятора QEMU.

**Задачи:**

1. Реализовать функцию `putchar` вывода данных в консоль.
2. Реализовать функцию `getchar` для получения данных из консоли.
3. На базе реализованных функций `putchar` и `getchar` написать программу, позволяющую вызывать определённые варианты функции OpenSBI посредством взаимодействия пользователя через меню: a. 1 — Get SBI implementation version b. 2 — Hart get status c. 3 — Hart stop d. System Shutdown

## 2. Ход выполнения

Для всех реализованных функций будем переиспользовать функцию `sbi_call`:

```
static struct sbiret sbi_call(long arg0, long arg1, long arg2, long
arg3,
                                long arg4, long arg5, long fid, long
eid) {
    register long a0 __asm__("a0") = arg0;
    register long a1 __asm__("a1") = arg1;
    register long a2 __asm__("a2") = arg2;
    register long a3 __asm__("a3") = arg3;
    register long a4 __asm__("a4") = arg4;
    register long a5 __asm__("a5") = arg5;
    register long a6 __asm__("a6") = fid;
    register long a7 __asm__("a7") = eid;

    __asm__ __volatile__("ecall"
                        : "+r"(a0), "+r"(a1)
                        : "r"(a2), "r"(a3), "r"(a4), "r"(a5), "r"(a6),
"r"(a7)
                        : "memory");

    struct sbiret ret;
    ret.error = a0;
    ret.value = a1;
    return ret;
}
```

### 2.1. putchar

```
void putchar(int ch) { sbi_call(ch, 0, 0, 0, 0, 0, 0, 0x01); }
```

Функция осуществляет вывод одиночного символа в консоль. Реализована через обёртку `sbi_call`, которая вызывает расширение Console Putchar (EID 0x01).

## 2.2. getchar

```
int getchar(void) {
    struct sbiret ret;
    do {
        ret = sbi_call(0, 0, 0, 0, 0, 0, 0, 0x02);
    } while (ret.error < 0);
    return (int)ret.error;
}
```

Функция осуществляет чтение одиночного символа с устройства ввода. Функция работает в режиме блокирующего ожидания (цикл do-while), вызывая расширение Console Getchar (EID 0x02) до тех пор, пока в регистре возврата не появится код символа (значение, не меньшее нуля). Возвращает считанный байт в формате int.

## 2.3. Функции по варианту

### 2.3.1. Get SBI implementation version

```
void cmd_get_impl_version(void) {
    struct sbiret ret = sbi_call(0, 0, 0, 0, 0, 0, 2, 0x10);
    puts("SBI implementation version: ");
    print_dec(ret.value);
    puts(" (");
    print_hex(ret.uvalue);
    puts(")\n");
}
```

Запрашивает версию реализации SBI, поддерживаемую текущим окружением. Использует базовое расширение (EID 0x10, FID 2). Полученное значение из регистра a1 выводится в десятичном и шестнадцатеричном виде.

### 2.3.2. Hart get status

```
void cmd_hart_get_status(void) {
    puts("Enter hart id: ");
    int hartid = read_int();
    struct sbiret ret = sbi_call(hartid, 0, 0, 0, 0, 0, 2, 0x48534D);
    if (ret.error != SBI_SUCCESS) {
        puts("Error: ");
        puts(sbi_err_str(ret.error));
        putchar('\n');
        return;
    }
    puts("Hart ");
    print_dec(hartid);
    puts(" status: ");
    switch (ret.value) {
        case 0: puts("STARTED"); break;
        case 1: puts("STOPPED"); break;
        /* ... */
    }
    putchar('\n');
}
```

Запрашивает статус указанного аппаратного потока (hart). Для ввода идентификатора hart применяется вспомогательная функция `read_int`. Использует расширение HSM (EID 0x48534D, FID 2). Возможные значения статуса: `STARTED`, `STOPPED`, `START_PENDING`, `STOP_PENDING`, `SUSPENDED`, `SUSPEND_PENDING`, `RESUME_PENDING`.

### 2.3.3. Hart stop

```
void cmd_hart_stop(void) {
    puts("Stopping current hart...\n");
    struct sbiret ret = sbi_call(0, 0, 0, 0, 0, 0, 1, 0x48534D);
    puts("Error: ");
    puts(sbi_err_str(ret.error));
    putchar('\n');
}
```

Останавливает текущий hart. Использует расширение HSM (EID 0x48534D, FID 1). При успешном вызове управление не возвращается в программу; в противном случае выводится код ошибки SBI.

### 2.3.4. System Shutdown

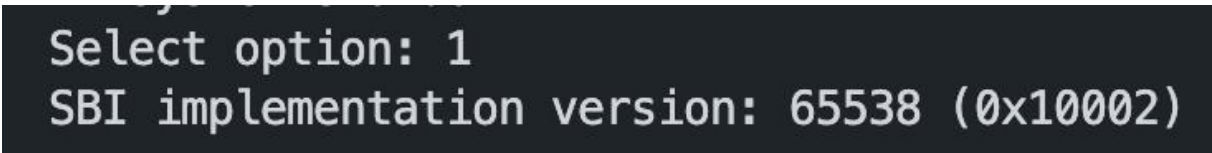
```
void cmd_shutdown(void) {  
    puts("System shutdown...\n");  
    sbi_call(0, 0, 0, 0, 0, 0, 0, 0x53525354);  
    puts("Shutdown failed!\n");  
}
```

Завершает работу виртуальной машины при помощи системного вызова расширения System Reset (EID 0x53525354, FID 0, тип сброса — shutdown). Управление после успешного вызова не возвращается; при неудаче выводится сообщение об ошибке.

### 3. Результаты работы программы

Ниже приведены скриншоты вывода в консоль при вызове каждого пункта меню.

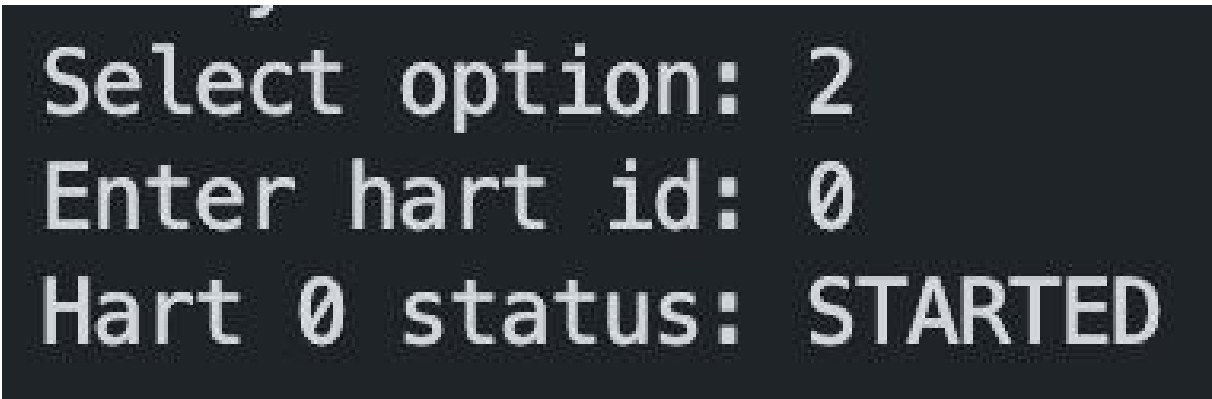
#### 3.1. Get SBI implementation version



```
Select option: 1  
SBI implementation version: 65538 (0x10002)
```

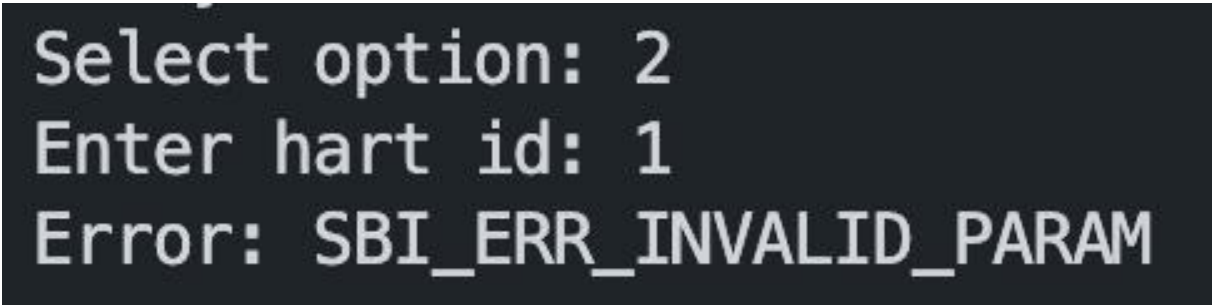
Рис. 1. Пункт меню 1 — Get SBI implementation version

#### 3.2. Hart get status



```
Select option: 2  
Enter hart id: 0  
Hart 0 status: STARTED
```

Рис. 2. Пункт меню 2 — Hart get status (hart id = 0)



```
Select option: 2  
Enter hart id: 1  
Error: SBI_ERR_INVALID_PARAM
```

Рис. 3. Пункт меню 2 — Hart get status (некорректный hart id)

#### 3.3. Hart stop



```
Select option: 3  
Stopping current hart...
```

Рис. 4. Пункт меню 3 — Hart stop



### 3.4. System Shutdown

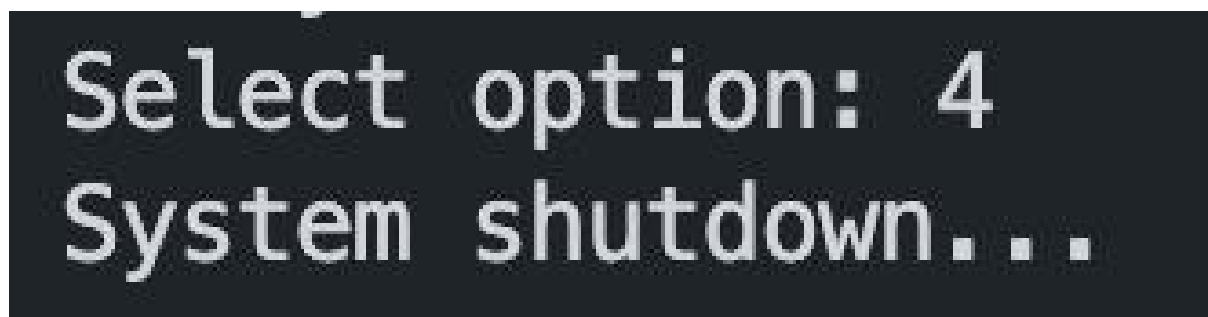


Рис. 5. Пункт меню 4 — System Shutdown